Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Seminer sunumları
  - Bu ders öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırmayla ilgili yayımlanmış bir makaleyi anlattı; hoca sunumdan sonra sorular sordu ve konuyu derste görülen kavramlara bağladı.
  - Beş sunum yapıldı: DNN ile tahmin tabanlı sismik veri sıkıştırma, dil modelleriyle metin sıkıştırma, doğal dil işleme için Bayesçi sıkıştırma (RNN seyreltme), evrişimli otokodlayıcılarla sismik veri sıkıştırma ve Draco kütüphanesinin ROS'a entegrasyonuyla 3B örgü ve nokta bulutu sıkıştırma.
- Sunum 1: Derin sinir ağı (DNN) ile tahmin tabanlı sismik veri sıkıştırma
  - Sismik veriler petrol ve gaz araması için karada ve denizde farklı konumlara yerleştirilen sensörlerle toplanır. Hem büyük depolama alanı hem de işleme merkezlerine iletim için yüksek bant genişliği ister.
  - Tahmin tabanlı yaklaşım: $n+1$. örnek, kendinden önceki n örnekten tahmin edilir. Orijinal değer ile tahmin arasındaki farka **residual (fark) değer** denir. Fark değerleri nicemlenir ve Huffman ile kodlanır; tahmin parametreleri başlıkta gönderilir. Alıcı aynı tahmini yapar, Huffman kodunu çözüp farkı ekleyerek orijinali bulur.
  - LPC (Linear Predictive Compression), otoregresif modelle önceki değerlerden **doğrusal** bir tahmin yapar. DNN ise birden çok gizli katmanla **doğrusal olmayan** bir tahmin yapar; tahmin daha hassas olduğu için fark değerlerinin varyansı küçülür.
  - Restricted Boltzmann Machine (RBM): iki katman arasındaki düğümler ilişkilidir, aynı katmandaki düğümler arasında bağlantı yoktur. Çalışmada DNN'in başlangıç ağırlıklarını optimuma yakın seçmek için RBM kullanıldı (ekrandan, 00:20:00).
  - Aşamalar (ekrandan, 00:20:00): DNN ve RBM ile eğitim, denetimli yöntemlerle tahmin değerlerinde ince ayar, fark değerlerinin nicemlenmesi, değişken kod uzunluğu için Huffman kodlama.
  - Sonuçlar: DNN bütün konfigürasyonlarda LPC'den daha yüksek SNR verdi (ekrandan, 00:21:50). Fark değerlerinin Huffman ile kodlanması, sabit uzunluklu (uniform) kodlamadan daha yüksek sıkıştırma oranı verdi (ekrandan, 00:22:30).
- Sunum 2: Dil modelleriyle metin sıkıştırma
  - Geleneksel sıkıştırma algoritmaları kelimelerin birbirini izleme olasılığını kullanmaz. Makale, bir dil modelinin olasılık tahminlerini kod kelimeleri üretmekte kullanır.
  - Algoritma (ekrandan, 00:34:40): dil modeli eğitilir (unigram, bigram, trigram olasılıkları); her bigram için üçüncü kelimenin bir kelime ya da UNK olma olasılığı hesaplanır; her bigram için üçüncü kelimenin dağılımına göre bir Huffman ağacı kurulur; ayrıca bir unigram Huffman ağacı kurulur; kelimeler ilgili ağaçtaki bitlerle değiştirilir.
  - Bilinmeyen kelime UNK ile kodlanır ve daha düşük bir modele geçilir; unigram da tanımazsa karakter sıkıştırmaya inilir.
  - Kayan bağlam penceresi (sliding context window): pencere içindeki kelimelere bakarak belgenin konusu ya da dili tahmin edilir ve o kategoriye ait sözlük kullanılır (ekrandan, 00:40:00).
  - Perplexity, dil modelinin tahmin başarısını ölçer ve entropiyle ilişkilidir. Eğitim verisindeki trigram sayısı arttıkça perplexity düşer ve sıkıştırma iyileşir (ekrandan, 00:41:57). Sunumda 0,36 sıkıştırma oranıyla gzip ve bzip2 ile rekabet edilebildiği söylendi.

- Sunum 3: Doğal dil işleme için Bayesçi sıkıştırma (Bayesian Compression for NLP)
  - Amaç, çok parametreli RNN'leri sınırlı belleği olan cihazlarda kullanılabilecek kadar küçültmektir. Önceki budama (pruning) yöntemleri sezgiseldir ve zaman alıcı hiperparametre ayarı ister.
  - İlgili çalışmalar üç gruptadır: ağırlık matrislerinin tahmini, ağırlık hassasiyetinin azaltılması (nicemleme) ve ağırlık matrislerinin seyreltilmesi (pruning).
  - Sparse VD: ağırlıklar rastgele değişken kabul edilir; log-uniform önsel dağılım ve faktörize normal sonsal yaklaşım kullanılır; varyasyonel alt sınır optimize edilir (ekrandan, 01:00:00). Sinyal-gürültü oranı düşük ağırlıklar sıfırlanıp atılır.
  - Sparse VD-Voc: gömme katmanına çarpımsal ağırlıklar eklenir; sıfırlanan ağırlıklar gereksiz kelimelerin sözlükten atılması demektir.
  - Deneyler: metin sınıflandırma (IMDb, AGNews) ve dil modelleme (Penn Treebank, karakter ve kelime düzeyi). Metin sınıflandırmada çok yüksek sıkıştırma oranlarına doğruluk kaybı olmadan ulaşıldı; dil modellemede oranlar daha düşük kaldı (ekrandan, 01:05:30).
- Sunum 4: Evrişimli otokodlayıcı ile sismik veri sıkıştırma
  - Model 1, kodlayıcı (evrişim ve havuzlama katmanları) ve kod çözücüden (evrişim ve büyük olasılıkla yukarı örnekleme katmanları) oluşan bir evrişimli otokodlayıcıdır. Model 2 evrişim, düzleştirme (flatten) ve yoğun (dense) katmanlardan oluşan farklı bir CNN mimarisidir.
  - Ölçütler: NMSE, NRMSE ve SNR. Model 2, orta ve yüksek sıkıştırma oranlarında ELM ve DWT yöntemlerinden daha yüksek SNR verdi (ekrandan, 01:40:00).
  - Model 1'de en iyi SNR beş katmanla, Model 2'de iki katmanla elde edildi (ekrandan, 01:47:20).
- Sunum 5: Draco ile ROS'ta 3B örgü ve nokta bulutu sıkıştırma
  - Uzaktan kumandalı kurtarma robotlarında operatörle robot arasındaki bant genişliği sınırlıdır. Google'ın açık kaynaklı geometri sıkıştırma kütüphanesi Draco, ROS'a (Robot Operating System) entegre edildi.
  - LIDAR ışıkla mesafe ölçer; RGB-D kameralar derinlik ölçer. Örgü (mesh) köşe, kenar ve yüzeylerden; nokta bulutu bağlanmamış koordinat noktalarından oluşur.
  - Nokta bulutlarında çoğunlukla octree (bir küpü 8 alt küpe bölen ağaç) kullanılır; ağaç derinliği bant genişliğine göre sınırlanabilir. Draco örgüleri Edgebreaker ile (yüzeyleri spiral biçimde gezip bağlantıyı bir string'e dökerek), nokta bulutlarını k-d ağacıyla sıralamaya göre optimize edilmiş kodlamayla sıkıştırır.
  - Sıkıştırmayı esas olarak niceleme bit sayısı belirler. Bit sayısı azaldıkça veri küçülür ama ayrıklaştırma hataları artar; 25 bitte geometrik hata ihmal edilebilir düzeydedir.
  - Kodlama süreleri sensörün kayıt hızının altındadır, yani sıkıştırma gerçek zamanlı yapılabilir (ekrandan, 02:18:45).

Hocanın Özellikle Vurguladığı Kısımlar

- Residual (fark) sıkıştırma derste görülen bir fikirdir
  - Hoca, daha önceki bir derste 7-8 farklı sembolü farkları alarak 1 ve -1 gibi iki sembole indirdiği örneği hatırlattı. Sismik veride de aynı fikir uygulanır.
  - Bir deprem kaydında büyük salınım olan kısımlarda farkların varyansı büyür. Bu yüzden verinin tamamı tek blok olarak değil, küçük bloklara bölünerek sıkıştırılmalıdır; sunumdaki çalışmada da veri $N = 5, 10, 15$ segment değerleriyle parçalara bölündü.
- Varyans, entropi ve sıkıştırma ilişkisi
  - Fark değerlerinin varyansı ne kadar küçükse entropi o kadar düşük, ulaşılabilecek sıkıştırma oranı o kadar yüksek olur.
- Sıkıştırma oranı ile sıkıştırma performansı farklı ifadelerdir
  - Sıkıştırma oranı 0,36 ise, sıkıştırma performansı (kazanç) $1 - 0{,}36 = 0{,}64$, yani %64'tür.
- Kelime tabanlı Huffman ağacının boyu
  - Hoca, unigram, bigram ve trigramların aynı ağaca konmasının ağacı çok derinleştireceğini, entropi verilmeden ortalama bit sayısının ve ağaç derinliğinin kestirilemeyeceğini söyledi. Belge paragraf paragraf işlenirse ağaç o kadar derin olmaz.
  - gzip ve bzip2 de Huffman kullanır; dil modeli denen n-gram'lar karakter ya da kelime tabanlı olabilir.
- Konu değişiminin sezilmesi

  - Kayan pencereye gelen yeni bir kelime penceredeki hiçbir kelimeyle örtüşmüyorsa, bu kelime daha önce geçmemiştir; demek ki konu (bağlam) değişmiştir.
- Sıkıştırma oranının yazılışı
  - "1:100" gibi bir oran, 100 bitin 1 bite sıkıştırıldığı anlamına gelir. Tabloda yalnız "100" yazıldığında da aynı anlam kastedilir.
- RMS hata ile SNR ters orantılıdır
  - Hoca, RMS hata (orijinal ile sıkıştırılıp açılmış veri arasındaki fark) büyüdüğünde SNR'nin küçüldüğünü, birinin küçük olduğu yerde diğerinin büyük olması gerektiğini vurguladı. SNR, görüntü sıkıştırmada da orijinal ile açılmış görüntü arasındaki bozulmayı ölçmek için kullanılmıştı.
  - Sıkıştırma oranı arttıkça SNR düşer, hata artar; CR, SNR ve MSE aynı eğilimi farklı açılardan gösterir.
- 3B veride sıkıştırma ve video benzetmesi
  - Nokta bulutunda girdi, noktaların koordinat bilgileridir ve sıkıştırma bu bilgilere uygulanır.
  - Ortamda hareketli nesneler olduğunda, videoda kareler arasındaki farkların kodlanmasına benzer bir yaklaşım düşünülebilir. Hareketsiz bir nesne bir kez tanımlandıktan sonra daha az bitle, hareketli nesneler farklı bit sayılarıyla kodlanabilir; bu, sembollerin frekansına göre kod verilmesine benzer.

Kısa Tekrar Notları

- Tahmin tabanlı sıkıştırma: önceki örneklerden tahmin, fark (residual) değeri, nicemleme, Huffman ile kodlama. Alıcı aynı tahmini yapıp farkı ekler.
- LPC doğrusal, DNN doğrusal olmayan tahmin yapar. DNN farkların varyansını küçültür; düşük varyans düşük entropi ve yüksek sıkıştırma demektir.
- Veri bloklara bölünerek sıkıştırılmalıdır; büyük salınımlı kısımlar farkların varyansını büyütür.
- Fark değerlerinde Huffman, sabit uzunluklu kodlamadan daha yüksek sıkıştırma oranı verir.
- Dil modeliyle sıkıştırma: bigram bağlamına göre ayrı Huffman ağaçları; bilinmeyen kelimede UNK ile alt modele, en sonda karakter kodlamaya iniş.
- Kayan bağlam penceresi konu ya da dil değişimini sezip uygun sözlüğe geçer.
- Perplexity düştükçe model daha iyi tahmin eder ve sıkıştırma iyileşir.
- Sıkıştırma oranı 0,36 ise kazanç %64'tür.
- Sparse VD: Bayesçi seyreltme ile RNN ağırlıklarının çoğu sıfırlanır; Sparse VD-Voc ayrıca gereksiz kelimeleri sözlükten atar.
- Otokodlayıcı: kodlayıcı özellik çıkarıp sıkıştırır, kod çözücü yeniden kurar. RMS hata ile SNR ters orantılıdır.
- Draco: örgüde Edgebreaker, nokta bulutunda k-d ağacı; sıkıştırma ve doğruluğu niceleme bit sayısı belirler.

Detaylı Açıklamalar

- **Tahmin tabanlı sismik veri sıkıştırma.** Yöntem iki aşamalıdır. İlk aşamada verinin bir kısmı, özellikle ilk n değer, tahmin modelini eğitmek için kullanılır. Ardından her örnek için orijinal değer ile tahmin arasındaki fark hesaplanır. Fark değerleri nicemlenir (ayrıklaştırılır) ve Huffman gibi bir algoritmayla ikili koda çevrilir. Tahmin modelinin parametreleri sıkıştırılmış verinin başlığına konur. Alıcı bu parametrelerle aynı tahmini üretir, Huffman kodunu çözerek farkları bulur ve tahminle farkı toplayarak orijinale ulaşır. Farklardan geri dönülebilmesi sıkıştırmanın şartıdır. LPC, önceki değerlerden doğrusal bir tahmin yaptığı için karmaşık verilerde sıkıştırma oranı düşük kalabilir ve kayıplı kullanımda geri çözmede hata payı artar. DNN doğrusal olmayan ilişkileri de yakaladığı için farkların varyansı sıfıra yakın çıkar. Sunumda gösterilen sonuçlarda DNN'in SNR değerleri yaklaşık 42-59 dB, LPC'ninkiler yaklaşık 23-30 dB aralığındaydı (ekrandan, 00:21:50). Fark değerleri Huffman ile kodlandığında sıkıştırma oranı nicemleme seviyesine göre yaklaşık 7,9 ile 14,4 arasında, sabit uzunluklu kodlamada ise 5,1 ile 7,8 arasında kaldı (ekrandan, 00:22:30). Çalışma kayıpsıza yakın kayıplı bir sıkıştırmadır: yeniden çözülen sinyal orijinale çok yakın çıkar.

- **Dil modelleriyle metin sıkıştırma.** Bir insan "will have to" kalıbının "will have it" kalıbından daha sık geçtiğini tahmin edebilir; dil modeli de bu tür bilgiyi olasılık olarak verir. Karakter düzeyinde dil modelinin getirdiği fark küçüktür; asıl kazanç kelime düzeyindedir. Her bigramın ardından gelen üçüncü kelimenin dağılımına göre ayrı bir Huffman ağacı kurulduğu için, bir bağlamda çok olası olan kelime o bağlamda kısa kod alır. Belge birden çok konu ya da dil içeriyorsa (örneğin İngilizce bir metinde Almanca alıntılar), kayan bağlam penceresi kategoriyi sezer ve o kategoriye ait sözlüğe geçer; böylece yabancı kelimeler için uzun kodlar harcanmaz.
- **Bayesçi sıkıştırma (Sparse VD).** Standart RNN'de gömme $\tilde{x}_t = w^e_{x_t}$, yinelemeli adım $h_{t+1} = \sigma(W^h h_t + W^x \tilde{x}_{t+1} + b^r)$, çıkış $\hat{y}_t = \text{softmax}(W^d h_t + b^d)$ biçimindedir; ağırlıklar $\omega = \{W^e, W^x, W^h, W^d\}$, sapmalar $B = \{b^r, b^d\}$ ile gösterilir ve sapmalar seyreltmeye dahil edilmez (ekrandan, 00:57:05). Her ağırlığa log-uniform önsel $p(w_{ij}) \propto 1/|w_{ij}|$ konur, sonsal $q(w_{ij} \mid \theta_{ij}, \sigma^2_{ij}) = \mathcal{N}(\theta_{ij}, \sigma^2_{ij})$ ile yaklaşılır ve veri olabilirliği ile KL ıraksamasının toplamı en küçüklenir (ekrandan, 01:00:00). Yöntemin sözel özeti: ağırlıklar örneklenir, RNN uygulanır, gradyan hesaplanır, testte ortalama ağırlıklar kullanılır; θ bileşenlerinin çoğu sıfıra yaklaşır ve sinyal-gürültü oranı düşük ağırlıklar atılır. Sonuç tablosunda IMDb için Sparse VD 1135 kat, Sparse VD-Voc 12985 kat sıkıştırma verdi ve sözlük 20000 kelimeden sırasıyla 4611 ve 292 kelimeye indi; dil modellemede sıkıştırma kelime düzeyinde 14 kat civarında kaldı (ekrandan, 01:05:30). Hoca ve sunan öğrenci, verinin metin olduğunu, bu yüzden metnin kendisine kayıplı sıkıştırma uygulanamayacağını; burada sıkıştırılanın modelin parametreleri olduğunu tartıştı.
- **Evrişimli otokodlayıcıyla sismik veri sıkıştırma.** Sismik veri, ses dalgalarıyla yer altını değerlendirmek için toplanan jeofizik veridir (ekrandan, 01:30:00). Model 1'in tablosunda sıkıştırma oranı 2:1'den 100:1'e çıktıkça SNR yaklaşık 40,7 dB'den 20,3 dB'ye düşer, NMSE ve NRMSE artar; örneğin 10:1'de SNR yaklaşık 24,7 dB'dir (ekrandan, 01:40:20). Hoca bu tabloyu RMS hata ile SNR'nin ters orantısını göstermek için kullandı. Model 1'de altı katmanlı yapıda özellik haritası sayısı arttıkça SNR'nin keskin biçimde düştüğü görüldü; bunun nedeni sunumda açıklanmadı (ekrandan, 01:47:20).
- **Draco-ROS entegrasyonu.** Yazarların geliştirdiği örgü araçları (Mesh Tools) yazılımı, örgüleri standart ROS mesajlarıyla (örneğin köşe, normal ve yüzey bilgisini taşıyan örgü geometrisi mesajı) taşır. Bir örgü iki yoldan gönderilebilir: doğrudan görselleştirme arayüzüne (RViz) ya da önce Draco kodlayıcı düğümüne. Kodlayıcı düğüm mesajı sıkıştırma parametrelerine göre Draco veri yapısına çevirip `DracoCompressedData` mesajında bir bayt akışı üretir; kod çözücü düğüm bu akışı yeniden orijinal mesajlara çevirir (ekrandan, 02:10:00). Bu mesaj yalnız Draco verisini tutar, sıkıştırma ayarları gibi meta veriler gelecekte eklenebilir. Değerlendirmede geometrik hata, orijinal veri ile çözülen veri arasındaki metrik mesafeler CloudCompare ile ölçülerek bulundu (ekrandan, 02:16:30). Kod çözme örgülerde kodlamadan belirgin biçimde hızlıdır; nokta bulutlarında iki süre birbirine yakındır (ekrandan, 02:18:45).
  - **Ders sonrası not:** Sunumda Velodyne verisi için kodlama sürelerinin 3-8 saniye olduğu söylendi; slayttaki tabloda bu değerler 3,41-8,12 **milisaniyedir** (ekrandan, 02:18:45). 20 Hz tarama, her tarama için 50 ms demektir; 3-8 ms kodlama süresi bu sürenin altında kaldığı için sıkıştırma gerçek zamanlı yapılabilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.